Scheme/ Answer Key for Valuation

Scheme of evaluation (marks in brackets) and answers of problems/key

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

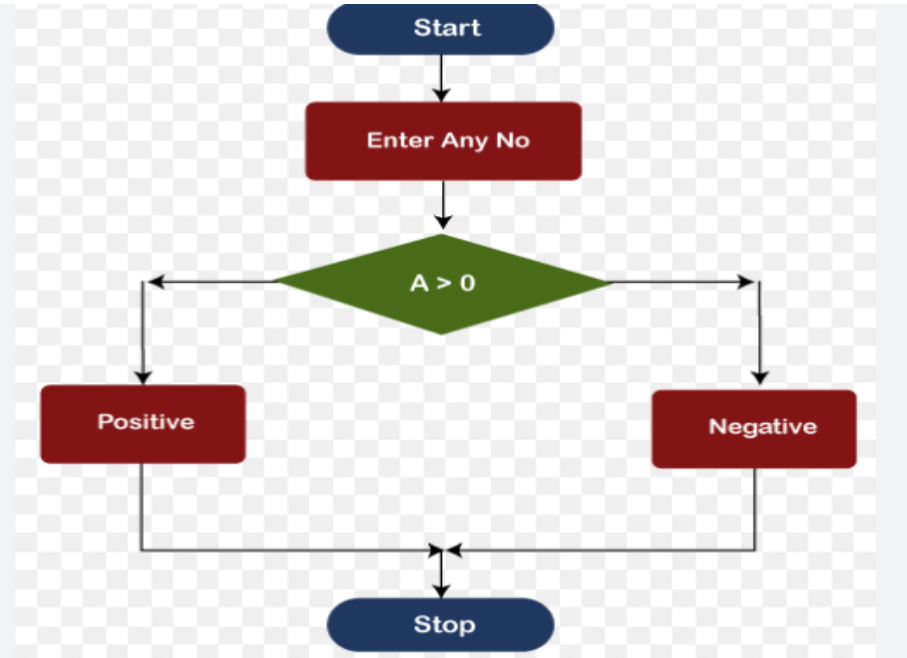
Second Semester B.Tech Degree Regular and Supplementary Examination June 2023 (2019 Scheme)

Course Code: EST 102

Course Name: PROGRAMMING IN C (Common to all programs)

Max. Marks: 100		Duration: 3 Hours

PART A

		Answer all Questions. Each question carries 3 Marks	Marks
1		The language processor that reads the complete source program written in high-level language as a whole in one go and translates it into an equivalent program in machine language is called a Compiler. The Assembler is used to translate the program written in Assembly language into machine code. The translation of a single statement of the source program into machine code is done by a language processor and executes immediately before moving on to the next line is called an interpreter.	(3)
2		A **flowchart** can also be defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task. the flow chart to check whether the given number is positive or negative (2 marks) 	(3)

3		A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.
		Syntax :
		`while (boolean condition)`
		`{`
		`  loop statements...`
		`}`
		do while loop is similar to while loop with the only difference that it checks for the condition after executing the statements, and therefore is an example of **Exit Control Loop.**
		Syntax:
		`do`
		`{`
		`  statements..`
		`}`
		`while (condition);`
			(3)

4		Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers. These I/O supports all data types like int, float, char, and many more. The following formatted I/O functions will be discussed in this section- 1. printf() 2. scanf() 3. sprintf() 4. sscanf()	(3)

5		***One-dimensional arrays***, also known as single arrays, are arrays with only one dimension or a single row. In this article, we'll dive deep into one-dimensional arrays in C programming language, including their ***syntax, examples***, and ***output***. ## Syntax of One-Dimensional Array in C The syntax of a ***one-dimensional array*** in C programming language is as follows: 1. dataType arrayName[arraySize];	(3)

6		`#include <stdio.h>`
		`int main() {`
		`  int num, originalNum, remainder, result = 0;`
		`  printf("Enter a three-digit integer: ");`
		`  scanf("%d", &num);`
		`  originalNum = num;`
		
		`  while (originalNum != 0) {`
		`  // remainder contains the last digit`
		`  remainder = originalNum % 10;`
		
		`  result += remainder * remainder * remainder;`
		
		`  // removing last digit from the orignal number`
		`  originalNum /= 10;`
		`  }`
		
		`  if (result == num)`
		`  printf("%d is an Armstrong number.", num);`
			(3)

```
    else
        printf("%d is not an Armstrong number.", num);

    return 0;
}
```

7 (3)

Actual Parameter

It is the actual value that is assigned to the process by a caller. Or in other words, we can say that it is the parameters that you determine when you invite the subroutine such as, Functions.

Formal Parameter

A formal parameter is a variable that you specify when you determine the subroutine or function. These parameters define the list of possible variables, their positions, and their data types

8 (3)

	STRUCTURE	UNION
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is **greater than or equal to the sum of sizes of its members.**	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of **union is equal to the size of largest member.**
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

9		Define scale factor (1 mark) Ex :- if p1 is an integer pointer with the initial value say 2800,then after the operations p1=p1+1,the value of p1 will be 2802,not 2801. when we increment pointer its value is increased by the length of the data type that it points to. This length is called scale factor. Scale factor for each data type (2 marks)	(3)
10		various modes (3 marks) ● `fopen()` - create a new file or open a existing file ● `fclose()` - close a file ● `getc()` - reads a character from a file ● `putc()` - writes a character to a file ● `fscanf()` - reads a set of data from a file ● `fprintf()` - writes a set of data to a file	(3)
PART B			
Answer any one Question from each module. Each question carries 14 Marks			
11	a) b)	Algorithm -5 marks Step 1 :Start Step 2 :Read a, b, c values Step 3 :Compute d = b2 4ac Step 4 :if d > 0 then r1 = b+ sqrt (d)/(2*a) r2 = b sqrt(d)/(2*a) Step 5 :Otherwise if d = 0 then	(14)

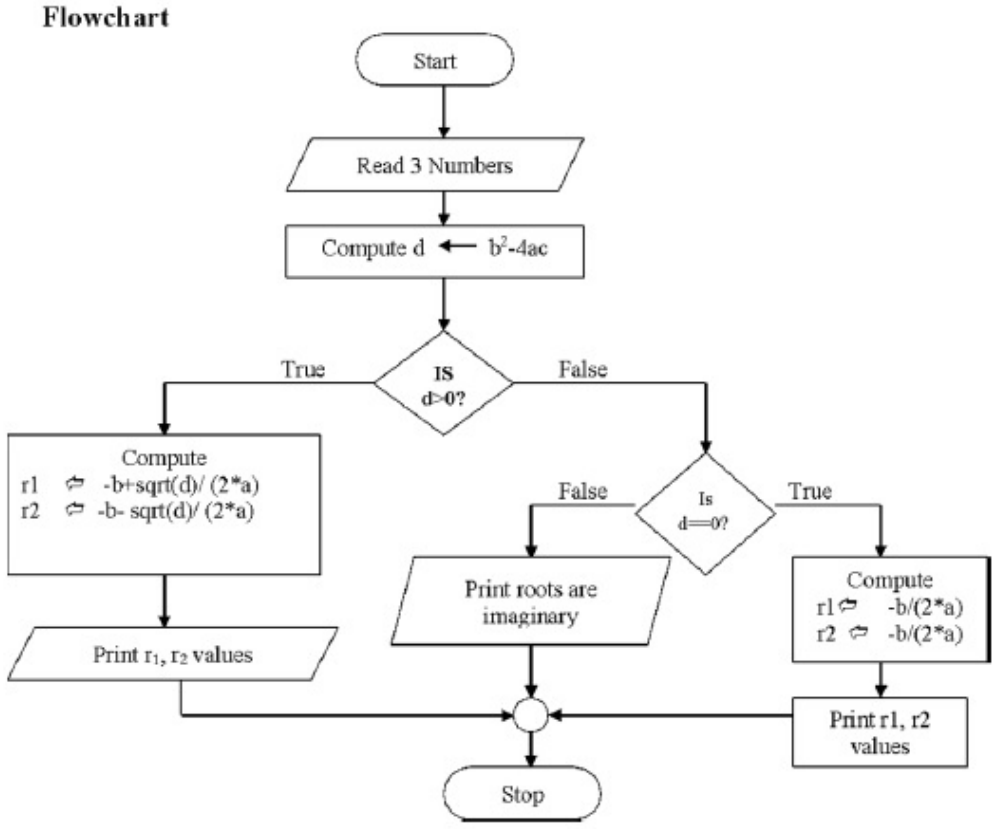
compute r1 = -b/2a, r2=-b/2a

print r1,r2 values

Step 6 :Otherwise if d < 0 then print roots are imaginary

Step 7 :Stop

Flowchart – 5 marks



Difference between system software and application software -3 marks

System Software	Application Software
Gives path to software application to run	Is designed and built for specific task
Is a general purpose software	Is a specific software
Low language is used	High language is used
Programming is complex when compared to application software	Programming is simple when compared to system software
Interacts with the hardware directly	Does not take the hardware into consideration, as it does not interact directly
Installed by the manufacturers	Installed by the users as needed
Works in the background	Works in the forefront
Is a packaged program	Can be said to be resources sharing client server
Example: Operating System, Programming Language, Communication Software, etc	Example: Word Doc, Spreadsheet, Database, Internet Explorer

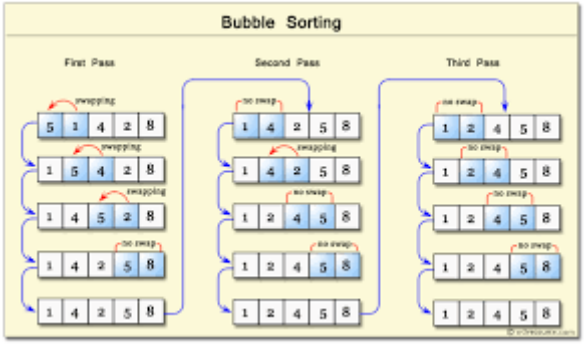
Any two examples for each -1 mark

OR

12 a) Algorithm - (5 marks) (14)

```
1. begin BubbleSort(arr)
2.    for all array elements
3.       if arr[i] > arr[i+1]
4.          swap(arr[i], arr[i+1])
5.       end if
6.    end for
7.    return arr
8. end BubbleSort
```

b) Explanation using example (5 marks)



Explain Memory hierarchy registers, cache memory primary and secondary memory (1 X 4= 4 marks)

Memory hierarchy refers to the organization and arrangement of different types of memory in a computer system based on their speed, capacity, and proximity to the CPU (Central Processing Unit). The primary purpose of memory hierarchy is to provide a balance between speed and capacity to ensure that data can be accessed quickly when needed while keeping costs and energy consumption in check.

Here are the key components of memory hierarchy, from fastest to slowest:

1. Registers:
   - Registers are the fastest and smallest type of memory in a computer system.
   - They are located directly in the CPU and are used to store data that the CPU is currently processing.
   - Registers can be accessed extremely quickly, but they have very limited capacity, typically just a few bytes.

2. Cache Memory:
   - Cache memory is a small and high-speed type of memory that sits between the CPU and main memory (RAM).
   - It is designed to store frequently accessed data and instructions to reduce the time the CPU spends waiting for data from slower memory.
   - Cache memory comes in multiple levels (L1, L2, and sometimes L3), with L1 being the closest to the CPU and the smallest but fastest, and L3 being larger but slower.
   - The cache uses a mechanism called caching algorithms to manage which data is stored in it, and it tries to keep the most relevant data for the CPU's current operations.

3. Primary Memory (RAM - Random Access Memory):
   - Primary memory, often referred to as RAM, is the main memory used for storing data and instructions that the CPU needs to perform its tasks.
   - RAM is faster than secondary memory but slower than cache memory.
   - It has a much larger capacity than cache memory and can store a substantial amount of data, but it is still volatile, meaning that data is lost when the computer is powered off.

4. Secondary Memory (Storage):
   - Secondary memory includes various types of non-volatile storage devices, such as hard disk drives (HDDs), solid-state drives (SSDs), optical drives, and more.
   - Secondary memory is significantly slower than primary memory and is used for long-term storage of data and programs.
   - It provides the ability to retain data even when the computer is powered off.
   - Data is typically transferred between secondary and primary memory when needed, and this transfer process is slower than accessing data directly from RAM.

In summary, the memory hierarchy in a computer system is designed to optimize the balance between speed and capacity, with registers and cache memory providing high-speed access to frequently used data and instructions,

		while primary memory (RAM) offers a larger but still relatively fast storage space. Secondary memory serves as long-term storage, albeit at a much slower speed compared to primary memory. The goal is to ensure that the CPU can access data quickly while managing the cost and energy efficiency of the system.	
13	a) b)	Program using switch – case—without syntax errors- 10 marks Explain usage of 'break' and 'continue' statements with proper example (2x2= 4 marks) • `break` is used to exit a loop prematurely when a specific condition is met. • `continue` is used to skip the rest of the current iteration and move to the next iteration of a loop when a certain condition is met. • Both `break` and `continue` statements provide control flow mechanisms within loops to customize the behavior based on specific conditions.	(14)
		OR	
14	a) b)	Listing of any 4 types of operators (1 mark) Explanation of each (1.5 x 4= 6 marks) Correct Program – 7 marks	(14)

15	a)	#include <stdio.h>
		
		#include <string.h>
		
		void concatenate_string(char* s, char* s1)
		
		{
		
		int i;
		
		int j = strlen(s);
		
		for (i = 0; s1[i] != '\0'; i++) {
			(14)

```
            s[i + j] = s1[i];
        }

        s[i + j] = '\0';

        return;
}

int main()
{

        char s[5000], s1[5000];

        printf("Enter the first string: ");
        gets(s);

        printf("Enter the second string: ");
        gets(s1);

        // function concatenate_string
        // called and s and s1 are
```

```
                    // passed
                    concatenate_string(s, s1);


                    printf("Concatenated String is: '%s'\n", s);


                    return 0;
}
```

b)

```
#include<stdio.h>
int main()
{
 int i, j, rows, columns, a[10][10], b[10][10], c = 1;

 printf("\nEnter the number of rows and columns  :  ");
 scanf("%d %d", &i, &j);

 printf("\nEnter the matrix elements \n");
 for(rows = 0; rows < i; rows++)
  {
        for(columns = 0;columns < j;columns++)
        {
```

```
    scanf("%d", &a[rows][columns]);
        }
 }
        //Finding out the Transpose of matrix
for(rows = 0; rows < i; rows++)
 {
        for(columns = 0;columns < j; columns++)
        {
    b[columns][rows] = a[rows][columns];
        }
 }

for(rows = 0; rows < i; rows++)
 {
        for(columns = 0; columns < j; columns++)
        {
   if(a[rows][columns] != b[rows][columns])
        {
        c++;
break;
}
```

		}
		}
		if(c == 1)
		{
		printf("\n The Matrix is a Symmetric Matrix. ");
		}
		else
		{
		printf("\n The Matrix is Not a Symmetric Matrix. ");
		}
		return 0;
		}	

		OR	

16	a)	**strlen()**
		strlen() function returns the length of the string. strlen() function returns integer value.
		Example:
		char *str = “Learn C Online”;
		int strLength;
		strLength = strlen(str); //strLength contains the length of the string i.e. 14
		strcpy()
			(14)

strcpy() function is used to copy one string to another. The Destination_String should be a variable and Source_String can either be a string constant or a variable.

Syntax:

strcpy(Destination_String,Source_String);

strncat()

strncat() is used to concatenate only the leftmost n characters from source with the destination string.

The Destination_String should be a variable and Source_String can either be a string constant or a variable.

Syntax:

strncat(Destination_String, Source_String,no_of_characters);

strcmp()

strcmp() function is use two compare two strings. strcmp() function does a case sensitive comparison between two strings. The Destination_String and Source_String can either be a string constant or a variable.

Syntax:

int strcmp(string1, string2);

This function returns integer value after comparison.

Value returned is 0 if two strings are equal.

If the first string is alphabetically greater than the second string then, it returns a positive value.

If the first string is alphabetically less than the second string then, it returns a negative value

b)

```
# include<stdio.h>
# include<math.h>
int main( )
{
int a[10][10], b[10][10], c[10][10];
int m,n,i,j,l,k ;
printf("\n Enter Order of A matrix :");
scanf("%d %d", &m, &n);
printf("Enter A matrix \n");
for( i=0; i<m; i++)
for ( j=0; j<n; j++)
scanf("%d", &a[ i ][ j ]);
printf("\n Enter order of B matrix:");
scanf("%d %d", &n, &l);
printf("Enter B matrix \n");
for( i=0; i<n; i++)
for ( j=0; j<l; j++)
scanf("%d", &b[ i ][ j ]);
/* loop to multiply two matrices */
for( i=0; i<m; i++)
{
for ( j=0; j<l; j++)
{
c[i ][ j ] = 0;
for( k=0; k < n; k++)
c[i ][ j ] = c[ i ][ j ] +a[ i ][ k ] * b[ k ][ j ];
}
}
printf("\n Resultant matrix is \n");
for( i=0; i<m; i++)
{
for( j=0; j<l; j++)
printf("%6d", c[ i ][ j ]);
printf("\n");
}

}
```

17	a) b)	Static ,auto ,extern, register—explain each(2x 4= 8 marks) Recursion – definition(1 mark) Correct program without syntax errors(5 marks) / Logically correct program with syntax errors(4 marks)	(14)
		OR	

18	a)	**Implement linear search using function. Reading the inputs and printing the result must be done in the main function.**
		#include <stdio.h>
		
		// Function to perform linear search
		int linearSearch(int arr[], int size, int target) {
		for (int i = 0; i < size; i++) {
		if (arr[i] == target) {
		return i; // Return the index where target is found
		}
		}
		return -1; // Return -1 if target is not found
		}
		
		int main() {
		int size, target;
		
		printf("Enter the size of the array: ");
		scanf("%d", &size);
		
		int arr[size];
		
		printf("Enter %d integers separated by spaces:\n", size);
		for (int i = 0; i < size; i++) {
		scanf("%d", &arr[i]);
		}
		
		printf("Enter the target value to search: ");
		scanf("%d", &target);
		
		// Call the linearSearch function to search for the target value
		int result = linearSearch(arr, size, target);
		
		if (result != -1) {
		printf("Target value %d found at index %d.\n", target, result);
		} else {
		printf("Target value %d not found in the array.\n", target);
		}
		
		return 0;
			(14)

	b)	} Explain user defined function with example – (2 marks) A user-defined function (UDF) is a custom function created by a programmer to perform a specific task or set of tasks within a program. UDFs allow you to encapsulate a block of code into a named function, making your code more modular, readable, and reusable. You can call a UDF multiple times from different parts of your program. **Explain user library function with example – (2 marks)** Library functions, also known as built-in functions, are functions that are provided as part of a programming language's standard library or external libraries. These functions are pre-defined and serve common purposes, making it easier for programmers to perform various tasks without having to implement everything from scratch. string_example = "Hello, World!" length = len(string_example) printf("Length of the string: %d", length) # Output: Length of the string: 13 # 2. max() and min() - Find the maximum and minimum values in an iterable	

19	a)	**Write a program to read and store the details (the name, employee code (integer) and salary) of 'n' employees in a company into a file using structure. Print the details of the employee whose employee code is given as input.**
		#include <stdio.h>
		#include <stdlib.h>
		
		// Define a structure to represent employee details
		struct Employee {
		char name[100];
		int empCode;
		float salary;
		};
		
		int main() {
		int n, searchCode;
		FILE *file;
		
		// Prompt the user to enter the number of employees
		printf("Enter the number of employees: ");
		scanf("%d", &n);
		
		// Create an array of Employee structures to store employee details
		struct Employee employees[n];
		
		// Input employee details
		for (int i = 0; i < n; i++) {
		printf("Enter details for Employee %d:\n", i + 1);
		printf("Name: ");
		scanf("%s", employees[i].name);
			(14)

```
    printf("Employee Code: ");
    scanf("%d", &employees[i].empCode);
    printf("Salary: ");
    scanf("%f", &employees[i].salary);
}

// Open a file for writing
file = fopen("employee_details.txt", "w");
if (file == NULL) {
    printf("Unable to create the file.\n");
    return 1;
}

// Write employee details to the file
for (int i = 0; i < n; i++) {
    fprintf(file, "Name: %s\n", employees[i].name);
    fprintf(file, "Employee Code: %d\n", employees[i].empCode);
    fprintf(file, "Salary: %.2f\n\n", employees[i].salary);
}

// Close the file
fclose(file);

// Prompt the user to enter an employee code for lookup
printf("\nEnter the Employee Code to lookup: ");
scanf("%d", &searchCode);

// Open the file for reading
file = fopen("employee_details.txt", "r");
if (file == NULL) {
    printf("Unable to open the file.\n");
    return 1;
}

int found = 0; // Flag to indicate if the employee is found
char line[100];

// Search for the employee with the given code
while (fgets(line, sizeof(line), file)) {
    if (sscanf(line, "Employee Code: %d", &employees[0].empCode) == 1) {
        if (employees[0].empCode == searchCode) {
            found = 1;
            printf("\nEmployee Found:\n");
            printf("%s", line); // Print the Employee Code
            fgets(line, sizeof(line), file); // Read the Name
            printf("%s", line);
            fgets(line, sizeof(line), file); // Read the Salary
            printf("%s", line);
            break; // Exit the loop once the employee is found
        }
    }
```

```
    }

    // If the employee is not found, print a message
    if (!found) {
        printf("\nEmployee with Employee Code %d not found.\n", searchCode);
    }

    // Close the file
    fclose(file);

    return 0;
}
```

OR

20 a) **Explain pass by reference using an example (3 marks)** (14)

Pass-by-reference means to pass the reference of an argument in the calling function to the corresponding formal parameter of the called function. The called function can modify the value of the argument by using its reference passed in.

```
#include <stdio.h>
swap (int *, int *);
 int main()
 {
int a, b;
printf("\nEnter value of a & b: ");
scanf("%d %d", &a, &b);
printf("\nBefore Swapping:\n");
printf("\na = %d\n\nb = %d\n", a, b);
swap(&a, &b);
printf("\nAfter Swapping:\n");
printf("\na = %d\n\nb = %d", a, b);
return 0; }
swap (int *x, int *y)
{
 int temp;
temp = *x;
*x = *y;
*y = temp; }
```

Correct program without syntax errors(6marks) / Logically correct program with syntax errors(4 marks)

b)

Write a program to copy the content of a file to another.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   FILE *sourceFile, *destinationFile;
   char sourceFileName[100], destinationFileName[100];
   char ch;

   printf("Enter the name of the source file: ");
   scanf("%s", sourceFileName);

   // Open the source file for reading
   sourceFile = fopen(sourceFileName, "r");
   if (sourceFile == NULL) {
      printf("Unable to open the source file.\n");
      return 1;
   }

   printf("Enter the name of the destination file: ");
   scanf("%s", destinationFileName);

   // Open the destination file for writing
   destinationFile = fopen(destinationFileName, "w");
   if (destinationFile == NULL) {
      printf("Unable to create/open the destination file.\n");
      fclose(sourceFile);
      return 1;
   }

   // Copy the contents of the source file to the destination file
   while ((ch = fgetc(sourceFile)) != EOF) {
      fputc(ch, destinationFile);
   }

   printf("File copied successfully.\n");

   // Close the files
   fclose(sourceFile);
   fclose(destinationFile);

   return 0;
}
```

****